VisionGuard: Deep Viva Questions

BACSE324 Embedded Systems Architecture. These go beyond "what does it do" into how the chip and the code really work. Learn the reasoning, not just the words.

1. Inside the ATmega328P (architecture)

Q1. What is the architecture of the ATmega328P?

An 8-bit AVR **RISC** microcontroller with a **Harvard architecture**: program memory (flash) and data memory (SRAM) are separate, with separate buses, so it can fetch the next instruction while executing the current one. It has **32 general-purpose registers**, and most instructions run in **one clock cycle**, so at 16 MHz it does close to 16 million instructions per second.

Q2. Harvard vs von Neumann: what's the difference, and why does it matter here?

Von Neumann uses one memory and bus for both code and data, so fetching and data access compete. Harvard separates them, allowing a simple 2-stage pipeline (fetch + execute) and faster, predictable timing, which is useful for real-time sensing.

Q3. What are the three memories, and what does each hold in your project?

Flash (32 KB): our program, plus a small bootloader. **SRAM (2 KB):** variables while running, like `distance` and `soilMoisture`, and the stack. **EEPROM (1 KB):** non-volatile data. We don't use it now, but it could store the calibrated water threshold so it survives power-off.

Q4. How does the code get from the laptop into the chip?

The Arduino IDE uses **avr-gcc** to compile the C++ code into machine code (a **.hex** file). **avrdude** sends it over USB; the **CH340** chip converts USB to serial (UART); the **bootloader** already in flash receives it and writes it into flash. Then the program starts from `setup()`.

Q5. What actually happens when you call digitalWrite()?

It sets a bit in the port's **PORTx register**. The pin direction is set in the **DDRx register** by `pinMode()`, and inputs are read from the **PINx register**. `digitalWrite()` takes a few microseconds because it does safety checks; writing the register directly takes a single clock cycle (62.5 ns at 16 MHz).

Q6. Is your system an RTOS-based system?

No. It's a **super-loop** (bare-metal) design: `setup()` runs once, then `loop()` repeats forever with no operating system. It's a **soft real-time** system: a slightly late alert is not catastrophic, but the response should be fast.

2. ADC (moisture reading)

Q7. What type of ADC does the ATmega328P have?

A **10-bit successive-approximation** ADC. It compares the input against a binary search of voltages, one bit at a time, from MSB to LSB. It has a multiplexer, so one ADC serves several analog pins (A0–A7 on the Nano).

Q8. What is the resolution, and how long does one reading take?

With a 5 V reference: 5 / 1024 ≈ **4.9 mV per step**. Arduino sets the ADC clock to 16 MHz / 128 = 125 kHz, and one conversion takes 13 ADC clocks, about **104 µs**.

Q9. What is the ADC reference, and why does it matter with a battery?

By default the reference is AVcc, the supply itself. Our supply is the battery (3.0–4.2 V), so the reference changes as it drains. But the moisture divider is fed from the same supply, so the reading is **ratiometric**: the ratio stays the same and the reading barely changes.

Q10. Why read A0 instead of the LM393's D0 output?

D0 is a 1-bit yes/no from the comparator, with its threshold set by the potentiometer. A0 gives the actual level (0–1023), so we can set and change the threshold in software, see it on the Serial Monitor and calibrate properly.

3. Timing and the ultrasonic sensor

Q11. How does pulseIn() work internally? What's the drawback?

It **busy-waits**: it waits for the pin to go HIGH, then counts in a loop until it goes LOW, and converts the count to microseconds. The drawback is that it **blocks** the CPU; nothing else runs while it waits (up to our 30 ms timeout).

Q12. How could you measure the echo without blocking?

Use **interrupts**: a pin-change interrupt records the time (`micros()`) on the rising and falling edges. Better still, use **Timer1 input capture** (pin ICP1 = D8), where the hardware timer captures the exact edge time. The CPU stays free between edges.

Q13. Why exactly 30 ms for the timeout?

The maximum range is 400 cm, so the round trip is 800 cm ÷ 0.034 cm/µs ≈ 23.5 ms. 30 ms covers that with margin (about 5 m). Without a timeout, pulseIn waits up to 1 second, freezing the cane.

Q14. What is the distance resolution in theory?

1 µs of echo time = 0.034 ÷ 2 = 0.017 cm ≈ 0.17 mm. In practice the sensor's accuracy is about ±3 mm, and our code stores whole centimetres (an int), so we get 1 cm resolution, which is enough for a cane.

Q15. Why can't the HC-SR04 measure below 2 cm?

After transmitting, the transducer keeps vibrating (**ringing**) for a short time. Echoes that return during this time can't be separated from the transmit pulse. This is the blind zone.

Q16. Why 40 kHz?

It's the resonant frequency of the transducers. It's above human hearing, and it's a compromise: higher frequency gives better resolution but is absorbed more by air (shorter range); lower frequency travels further but with poorer resolution and a wider beam.

Q17. How often should you ping? Does your code follow it?

The datasheet suggests a measurement cycle of **over 60 ms**, so old echoes don't confuse the next reading. Our loop delay is 30 ms plus the measurement time, which can be under 60 ms when the path is clear. Raising the delay to 60 ms is a planned fix.

4. Code efficiency and logic

Q18. The ATmega328P has no floating-point unit. Is `duration * 0.034 / 2` a problem?

Floating-point is done in software, which is slower and uses more flash. For our loop speed it's fine, but an integer version is better: `distance = duration / 58;` (because 1 ÷ 0.017 ≈ 58.8 µs per cm).

Q19. What is the worst-case response time to water?

If water appears just after an obstacle beep starts: 50 ms beep + up to 400 ms gap + 30 ms delay + up to 30 ms pulseIn ≈ **0.5 s**. That's because `delay()` blocks. Using `millis()`-based timing would make it check the sensors every few milliseconds.

Q20. Why use delay() if it blocks?

It's simple and easy to read for a first prototype, and 0.5 s is acceptable for walking speed. For the final version we'd switch to non-blocking `millis()` timing (or a timer interrupt) so sensing continues while beeping.

Q21. How does map() calculate the gap?

It's linear interpolation using integer maths: `(x − 2) × (400 − 50) / (50 − 2) + 50`. For x = 26 cm: (24 × 350 / 48) + 50 = 175 + 50 = 225 ms. Integer division drops the decimals.

Q22. How would you stop false alarms?

Average or take the median of 3–5 readings (a median filter removes single wrong echoes), and use **hysteresis**: for example, turn the water alarm on below 500 but off only above 550, so it doesn't flicker near the threshold.

5. Power and electrical

Q23. Can the ATmega328P run at 16 MHz on a 3.7 V battery?

Strictly by the datasheet's speed grades, 16 MHz needs about **3.8 V or more** (10 MHz is guaranteed from 2.7 V and 20 MHz from 4.5 V, roughly linear in between). A full cell (4.2 V) is fine, but as it drains below about 3.8 V we're outside the guaranteed range. It usually still works, but it isn't guaranteed. That's why a **5 V boost converter** is our first power upgrade.

Q24. What happens when the battery gets very low?

The **brown-out detector** (usually set at about 2.7 V on Arduino boards) resets the chip so it doesn't behave unpredictably. But Li-ion cells should not be discharged below about 3.0 V, and we have no protection circuit. A TP4056 module with protection (DW01) or a low-battery warning in code would fix this.

Q25. Is it safe to drive the buzzer directly from a pin?

A Nano pin is rated 40 mA absolute maximum, 20 mA recommended. Small active buzzers draw roughly 20–30 mA, so it works but is close to the limit. A transistor or MOSFET driver would be more reliable, and is required for the vibration motor.

Q26. Why does a motor need a MOSFET and a flyback diode?

A motor needs more current than a pin can give, so the pin switches the logic-level IRLZ44N MOSFET, which carries the motor current. A motor is an inductor: when switched off it produces a reverse voltage spike that can damage the MOSFET, and the **flyback diode** gives that current a safe path.

Q27. Why a 220 µF capacitor? What would a 0.1 µF add?

220 µF is a **bulk capacitor**: it supplies sudden current ($i = C \cdot dV/dt$) when the buzzer switches, preventing dips that could reset the Nano. A 0.1 µF ceramic capacitor near the chip filters **high-frequency** noise. They do different jobs, and using both is good practice.

Q28. Total power and battery life calculation?

About 40 mA × 3.7 V ≈ 0.15 W. The cell holds 4.44 Wh, so 4.44 ÷ 0.15 ≈ 30 hours in theory. In practice it's less, because of the conversion losses we'd add with a boost converter and the usable capacity of the cell.

Q29. How could you make the battery last longer?

Use the ATmega's **sleep modes** between readings, switch off the module power LEDs, power the moisture probe from a pin only while reading, and lower the clock speed.

6. Sensor physics and design

Q30. Why does the moisture probe corrode, and how do you stop it?

A constant DC current through water causes **electrolysis**, which eats away the positive track. Fixes: power the probe from a digital pin only for a few milliseconds during each reading, use AC excitation, or use a **capacitive** sensor with no exposed metal.

Q31. Will pure (distilled) water be detected?

Not well. Pure water conducts very poorly; real-world puddles and mud contain dissolved salts, which is what makes them conductive. Our sensor measures conductivity, not "wetness" directly.

Q32. What makes this an embedded system and not just a circuit?

A dedicated microcontroller runs one fixed program, reads sensors, processes data and controls outputs in real time, inside the product, with limited memory and power.

> **When a question goes beyond what you know:** don't guess. Say *"Sir, I'm not fully sure about that detail. My understanding is ___, and I'd check the ATmega328P datasheet to confirm."* A clear partial answer with a method is better than a confident wrong one.